260319_2차 프로젝트_Unity 3기_우성혁

📌 **과제 제출 방법**

1. 전체 복사 후 개인페이지에 붙여넣은 뒤 사용합니다.
2. 과제 작성 후 우측 상단에 점3개 아이콘을 클릭한 뒤 [내보내기] > [PDF] 로 노션 페이지를 변환합니다.
   - 내보내기 형식: PDF
   - 데이터베이스 포함: 현재 보기
   - 포함할 콘텐츠: 모든 콘텐츠
3. 다운받은 zip 파일 압축 해제 후, PDF 파일 명을 `yymmdd_2차 프로젝트_Unity 3기_000` 로 변경합니다.
4. 과제 제출란에 PDF 파일을 제출합니다. (zip 파일이 아닌, **PDF 파일**을 제출해주세요!)

추후 포트폴리오, 프로젝트 문서에 이 과제들을 활용할 수 있으니 일일회고처럼 꼼꼼히 기록해주세요.

Scrum

**금일 진행한 스크럼 내용을 간략하게 요약해주세요*

**스크럼을 진행하지 않았다면 공란으로 둡니다.*

- 시나리오 진행 방향
  • 하나의 동화를 만들어서 각 에피소드에 동화 내 역할을 부여
  • 유저가 외부 독자처럼 플레이하다가 마지막에 자신이 잃었던 기억들을 찾았음을 깨닫게 됨
  - 스테이지1: 이름을 잃어버린 소녀 → 어린 시절 친하게 지냈던 친구
  - 스테이지2: 화가를 꿈꿨음 → 자신의 기억
  - 스테이지3: 음악에서 영감을 준 가까운 사람(연인?)

    - 스테이지4: 기억을 합치며 자신을 깨달음 → 자신의 인생을 지켜보는 느낌을 줌
- 플레이어 역할: 단순 독자 → 잃어버린 이야기를 찾아서 연결해주는 기록자

Project

**프로젝트 작업에서 본인이 기여한 부분에 대해 작성합니다.*
**코드파일이 있다면 스크린샷이나 노션 코드를 사용하여 덧붙여주세요.*
(예: ERD 이미지, Figma 이미지, 코드 스크린샷, Unity 스크린샷 등)

---

1. **오늘 진행할 작업**

스테이지 4 발판 퍼즐 구현

게임플레이 기획

2. **진행중인 작업**

스테이지 4 발판 초기 데이터 배치 구현

▼ PuzzleData // 발판 데이터 구조화 스크립트

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
namespace PuzzleInfo
{
    public class PuzzleCube
    {
        public int id;
        public int[] place = new int[2];
        public bool[] colorBool = new bool[3];
        public enum switchCondition { near, row, column,
color };
        public switchCondition cond;
        public int switchValue;
    }
}
```

▼ PuzzleCubeData // 스크립터블오브젝트 연결 스크립트

```
using System.Collections;
using System.Collections.Generic;
```

```
using UnityEngine;
using PuzzleInfo;
public class PuzzleCubeData : MonoBehaviour
{
    public PuzzleCubeObj cubeData;
}
```

▼ **PuzzleCubeObj // 스크립터블오브젝트 정의 스크립트**

```
using PuzzleInfo;
using System.Collections;
using System.Collections.Generic;
using Unity.VisualScripting;
using UnityEngine;
using static PuzzleInfo.PuzzleCube;
[CreateAssetMenu(fileName = "PuzzleCube", menuName = "Cr
eate CubeData", order = 1)]
public class PuzzleCubeObj : ScriptableObject
{
    public int[] place = new int[2];
    public bool[] colorBool = new bool[3];
    public switchCondition cond;
    public int value;
}
```

▼ **PuzzleInit // csv 파일 → 스크립터블오브젝트 데이터 입력 스크립트**

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using PuzzleInfo;
using System.IO;
using UnityEditor;
using System;
using UnityEngine.UI;
using System.Linq;
public class PuzzleInit : EditorWindow
{
    public TextAsset puzzle_stage4;
```

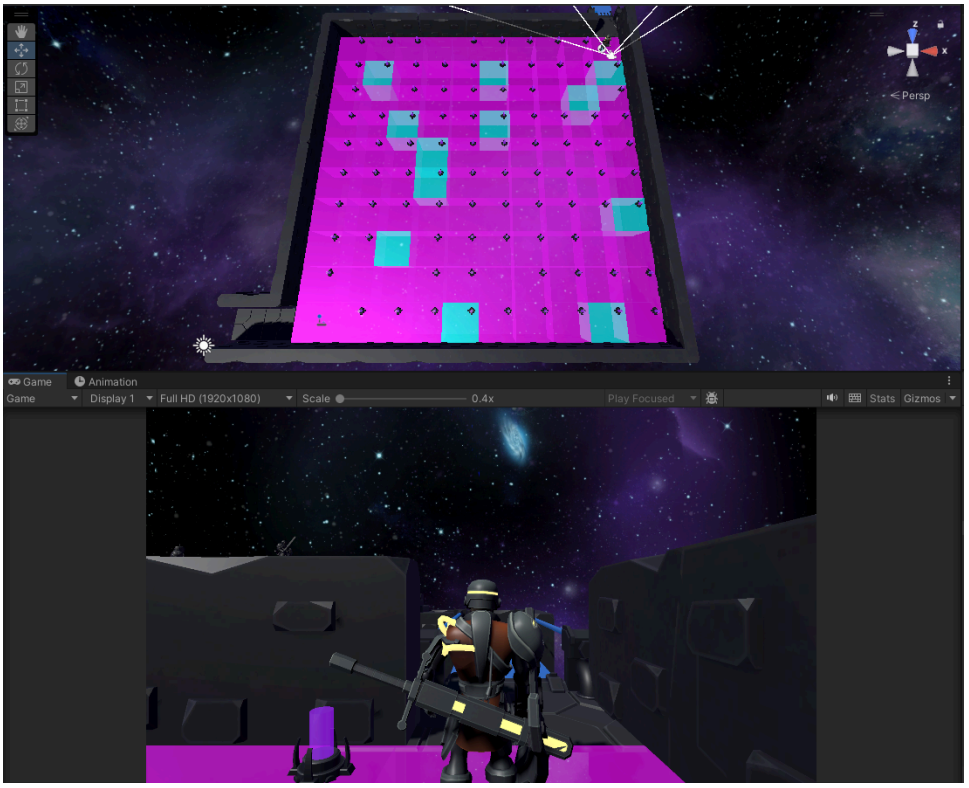
```
    public DefaultAsset cubeDataFolder;
    public PuzzleCubeObj[] targetObjects;
    private Vector2 scrollPos;
    [MenuItem("Tools/PuzzleCube CSV Importer")]
    public static void ShowWindow()
    {
        GetWindow<PuzzleInit>("PuzzleCube CSV Importe
r");
    }
    private void OnGUI()
    {
        scrollPos = EditorGUILayout.BeginScrollView(scro
llPos);
        puzzle_stage4 = Resources.Load<TextAsset>("csv/S
tage4_CubePuzzle");
        cubeDataFolder = (DefaultAsset)EditorGUILayout.O
bjectField("CubeData Folder", cubeDataFolder, typeof(Def
aultAsset), false);
        if (GUILayout.Button("Load Objects From Folder",
GUILayout.Height(25)))
        {
            LoadObjectsFromFolder();
        }
        EditorGUILayout.Space(10);
        if (targetObjects != null)
        {
            EditorGUILayout.LabelField($"Loaded Objects:
{targetObjects.Length}", EditorStyles.boldLabel);

            SerializedObject so = new SerializedObject(t
his);
            SerializedProperty arrayProp = so.FindProper
ty("targetObjects");
            EditorGUILayout.PropertyField(arrayProp, tru
e);
            so.ApplyModifiedProperties();
        }
        EditorGUILayout.Space(20);
```

```
        if (GUILayout.Button("Import CSV", GUILayout.Hei
ght(30)))
        {
            ImportCSV();
        }
        EditorGUILayout.EndScrollView();
    }
    private void LoadObjectsFromFolder()
    {
        if (cubeDataFolder == null)
        {
            Debug.LogError("폴더가 지정되지 않았습니다.");
            return;
        }
        string folderPath = AssetDatabase.GetAssetPath(c
ubeDataFolder);
        string[] guids = AssetDatabase.FindAssets("t:Puz
zleCubeObj", new[] { folderPath });
        targetObjects = guids
            .Select(guid => AssetDatabase.LoadAssetAtPat
h<PuzzleCubeObj>(AssetDatabase.GUIDToAssetPath(guid)))
            .OrderBy(obj => obj.name)
            .ToArray();
        Debug.Log($"{targetObjects.Length}개의 PuzzleCube
Obj를 로드했습니다.");
    }
    void ImportCSV()
    {
        if (puzzle_stage4 == null || targetObjects == nu
ll || targetObjects.Length == 0)
        {
            Debug.LogError("CSV 또는 ScriptableObject 배열
이 비어있습니다.");
            return;
        }
        string[] lines = puzzle_stage4.text.Split('\n');
        for (int i = 0; i < lines.Length; i++)
        {
```

```
            string line = lines[i].Trim();
            if (string.IsNullOrEmpty(line)) continue;
            string[] cols = line.Split(',');
            if (i >= targetObjects.Length)
            {
                Debug.LogWarning($"CSV 줄 {i}는 Scriptabl
eObject 배열 범위를 초과함");
                break;
            }
            PuzzleCubeObj obj = targetObjects[i];
            obj.place[0] = int.Parse(cols[1]);
            obj.place[1] = int.Parse(cols[2]);
            obj.colorBool[0] = cols[3].ToUpper() == "TRU
E";
            obj.colorBool[1] = cols[4].ToUpper() == "TRU
E";
            obj.colorBool[2] = cols[5].ToUpper() == "TRU
E";
            obj.cond = Enum.Parse<PuzzleCube.switchCondi
tion>(cols[6]);
            obj.value = int.Parse(cols[7]);
            EditorUtility.SetDirty(obj);
        }
        AssetDatabase.SaveAssets();
        Debug.Log("CSV 데이터를 PuzzleCubeObj에 성공적으로
덮어씌웠습니다.");
    }
}
```

스테이지 4 발판 색 변경 스위치 기능 구현

- 발판 오브젝트 내 스위치 트리거에 접촉 중인 상태에서 상호작용 버튼 입력 시 발동



▼ CubeSwitch // 스위치 작동 코드

```
using PuzzleInfo;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Linq;
using Unity.VisualScripting;
using UnityEngine;
public class CubeSwitch : MonoBehaviour
{
    private BoxCollider col;
    [SerializeField] private PuzzleCubeCtrl curCube;
    [SerializeField] private PuzzleCubeCtrl[] cubeList;
    [SerializeField] private List<PuzzleCubeCtrl> switch
Objects;
    public bool switchContact = false;
    private Char_Move user;
    private readonly string playerTag = "Player";
    void Awake()
```

```
    {
        col = GetComponent<BoxCollider>();
        cubeList = GameObject.Find("CubePuzzle").GetComp
onentsInChildren<PuzzleCubeCtrl>();
        curCube = GetComponentInParent<PuzzleCubeCtrl>
();
        switchObjects = new List<PuzzleCubeCtrl>();
        user = GameObject.FindGameObjectWithTag(playerTa
g).GetComponent<Char_Move>();
    }
    private void OnEnable()
    {
        user.Interact += UseSwitch;
    }
    private void OnTriggerEnter(Collider other)
    {
        if (curCube != null && other.gameObject.CompareT
ag("Player"))
        {
            switchContact = true;
            ConditionCheck();
        }
    }

    public void UseSwitch()
    {
        if (!switchContact) return;
        foreach (var obj in switchObjects)
        {
            obj.OnColorSwitch(curCube.switchValue);
        }
    }

    private void ConditionCheck()
    {
        switch (curCube.condition)
        {
            case PuzzleCube.switchCondition.near:
```

```
                switchObjects.Add(curCube);
                for (int n = 0; n < cubeList.Length; n+
+)
                {
                    if (Mathf.Abs(cubeList[n].column - c
urCube.column) + Mathf.Abs(cubeList[n].row - curCube.ro
w) == 1)  //이웃한 발판(전후좌우 1칸씩)
                    {
                        switchObjects.Add(cubeList[n]);
                    }
                }
                break;
            case PuzzleCube.switchCondition.row:
                for (int n = 0; n < cubeList.Length; n+
+)
                {
                    if (cubeList[n].row == curCube.row)
// 열 값이 같으면
                    {
                        switchObjects.Add(cubeList[n]);
                    }
                }
                break;
            case PuzzleCube.switchCondition.column:
                for (int n = 0; n < cubeList.Length; n+
+)
                {
                    if (cubeList[n].column == curCube.co
lumn)  // 열 값이 같으면
                    {
                        switchObjects.Add(cubeList[n]);
                    }
                }
                break;
            case PuzzleCube.switchCondition.color:
                for (int n = 0; n < cubeList.Length; n+
+)
                {
```

```
                    if (cubeList[n].cubeColor == curCub
e.cubeColor)  // 색상이 같으면
                    {
                        switchObjects.Add(cubeList[n]);
                    }
                }
                break;
        }
    }
    private void OnTriggerExit(Collider other)
    {
        if (other.gameObject.CompareTag("Player"))
        {
            for (int n = 0; n < cubeList.Length; n++)
            {
                switchObjects.Remove(cubeList[n]);
            }
            switchContact = false;
        }
    }
}
```

▼ **PuzzleCubeCtrl // 발판 관리 코드**

```
using PuzzleInfo;
using System;
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.Rendering;

public class PuzzleCubeCtrl : MonoBehaviour
{
    public GameObject cube;
    private Transform tr;
    public Rigidbody rb;
    public Color cubeColor;
    public GameObject colorSwitch;
    public PuzzleCubeData data;
```

```
    public int ID;
    public int row;
    public int column;
    public bool isRed;
    public bool isGreen;
    public bool isBlue;
    public PuzzleCube.switchCondition condition;
    public int switchValue;
    public event Action OnColorChanged;
    public int newHeight;
    public float switchTime = 1f;
    public void SetColor(Color color)
    {
        if (cubeColor != color)
        {
            cubeColor = color;
            OnColorChanged?.Invoke();
        }
    }
    void Awake()
    {
        data = GetComponent<PuzzleCubeData>();
        tr = GetComponent<Transform>();
        rb = GetComponent<Rigidbody>();
        CubeDataSetup();
    }
    void CubeDataSetup()
    {
        ID = data.cubeData.place[0] * 10 + data.cubeDat
a.place[1];
        row = data.cubeData.place[0];
        column = data.cubeData.place[1];
        isRed = data.cubeData.colorBool[0];
        isGreen = data.cubeData.colorBool[1];
        isBlue = data.cubeData.colorBool[2];
        condition = data.cubeData.cond;
        switchValue = data.cubeData.value;
    }
```

```
    void OnEnable()
    {
        transform.position = new Vector3(row * 2.5f, 0f,
column * 2.5f);
        cubeColor = new Color(isRed ? 1 : 0, isGreen ? 1
: 0, isBlue ? 1 : 0);
        cube.GetComponent<Renderer>().material.color = c
ubeColor;
    }
    void Start()
    {
        colorSwitch.gameObject.SetActive(switchValue !=
0);
        colorSwitch.GetComponent<BoxCollider>().enabled
= (switchValue != 0);
    }
    public void OnColorSwitch(int colorCode)  // colorCo
de: 1 for red, 2 for green, 4 for blue
    {
        if ((colorCode & 1) == 1) isRed = !isRed;
        if ((colorCode & 2) == 2) isGreen = !isGreen;
        if ((colorCode & 4) == 4) isBlue = !isBlue;
        cubeColor = new Color(isRed ? 1 : 0, isGreen ? 1
: 0, isBlue ? 1 : 0);
        cube.GetComponent<Renderer>().material.color = c
ubeColor;
        OnColorChanged?.Invoke();
    }
}
```

▼ NextCube // 이웃한 발판 색 감지 코드

```
using System.Collections;
using System.Collections.Generic;
using Unity.VisualScripting;
using UnityEngine;

public class NextCube : MonoBehaviour
{
```

```
    private BoxCollider col;
    private PuzzleCubeCtrl curCube;
    private PuzzleCubeCtrl[] cubeList;
    [SerializeField] private PuzzleCubeCtrl nextCube;
    public enum CubeDir { west, east, north, south }
    public CubeDir dir;
    private int nextX;
    private int nextY;
    private void Awake()
    {
        col = GetComponent<BoxCollider>();
        curCube = GetComponentInParent<PuzzleCubeCtrl>
();
        cubeList = GameObject.Find("CubePuzzle").GetComp
onentsInChildren<PuzzleCubeCtrl>();
    }
    private void Start()
    {
        switch (dir)
        {
            case CubeDir.west:
                nextX = curCube.row - 1;
                nextY = curCube.column;
                break;
            case CubeDir.east:
                nextX = curCube.row + 1;
                nextY = curCube.column;
                break;
            case CubeDir.north:
                nextX = curCube.row;
                nextY = curCube.column + 1;
                break;
            case CubeDir.south:
                nextX = curCube.row;
                nextY = curCube.column - 1;
                break;
        }
        for (int i = 0; i < cubeList.Length; i++)
```

```
        {
            if (cubeList[i].row == nextX && cubeList[i].
column == nextY)
            {
                nextCube = cubeList[i];
                break;
            }
        }
        curCube.OnColorChanged += UpdateCollider;
        if (nextCube != null) nextCube.OnColorChanged +=
UpdateCollider;
        UpdateCollider();
    }
    private void UpdateCollider()
    {
        if (nextCube != null)
            col.enabled = CanMoveThrough();
        else
            col.enabled = false;
    }
    public bool CanMoveThrough()
    {
        return (curCube.cubeColor != nextCube.cubeColo
r);
    }
}
```

▼ **Char_Move // 플레이어 캐릭터 조작 코드**

```
using System;
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.InputSystem;
public class Char_Move : MonoBehaviour
{
    private Transform tr;
    private Rigidbody rb;
    private Animator anim;
```

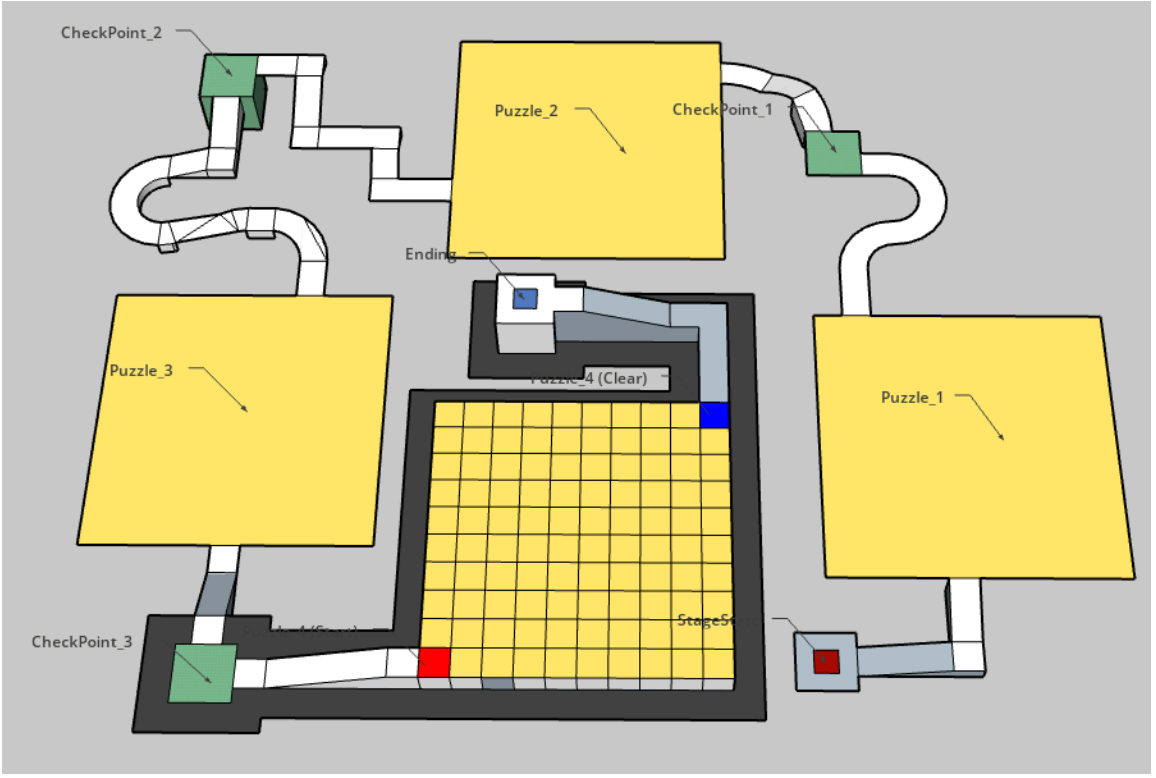
```
    private bool isRun = false;
    private readonly int hashW = Animator.StringToHash
("IsWalking");
    private readonly int hashR = Animator.StringToHash
("IsRunning");
    private Vector3 move = Vector3.zero;
    public float moveSpeed = 0f;
    public event Action Interact;
    void Awake()
    {
        tr = GetComponent<Transform>();
        rb = GetComponent<Rigidbody>();
        anim = GetComponent<Animator>();
    }
    private void FixedUpdate()
    {
        moveSpeed = isRun ? 5f : 2f;
        rb.MovePosition(rb.position + move * moveSpeed *
Time.deltaTime);
        if (move != Vector3.zero)
        {
            tr.rotation = Quaternion.LookRotation(move);
            if (!isRun)
            {
                anim.SetBool(hashW, true);
            }
        }
        else if (move == Vector3.zero)
        {
            tr.Rotate(Vector3.zero);
        }
        anim.SetBool(hashR, isRun);
    }
    public void OnMove(InputAction.CallbackContext conte
xt)
    {
        Vector2 input = context.ReadValue<Vector2>();
        move = new Vector3(input.x, 0, input.y);
```

```
        if(context.canceled)
        {
            anim.SetBool(hashW, false);
            move = Vector3.zero;
        }
    }
    public void OnRun(InputAction.CallbackContext contex
t)
    {
        if (context.performed)
        {
            isRun = true;
        }
        if (context.canceled)
        {
            isRun = false;
        }
    }
    public void OnInteract(InputAction.CallbackContext c
ontext)
    {
        Interact?.Invoke();
    }
}
```

3. **내일 개발 계획 및 목표**

스테이지 4 레벨 수정

- 각 발판에 동작 힌트 오브젝트 추가
  - 각 발판의 색 변경 조건 및 변경 색상을 표시
- 단위 발판 넓이(x,z) 증가 및 두께(y) 감소
  - 이웃 발판 색 감지 시 작동하는 이동 불가 콜라이더 및 스위치 상호작용 콜라이더의 배치 수정 필요
  - 높이 스위치를 사용하지 않게 되어 정육면체 형태를 유지할 필요가 없어짐
- 발판 퍼즐 이전 동선 공간 구성



- 스테이지 1, 2, 3에서 사용한 종류의 퍼즐을 1종류씩 추가로 배치
  - 각 퍼즐을 스테이지 순서대로 진행한 후 발판 퍼즐로 마무리하는 선형 구성
  - 각 퍼즐 사이에 체크포인트를 배치하여 로비로 이동할 수 있게 함
  - 스토리 개요 설정에 맞추어 엔딩 지점이 신 중앙에 위치하도록 구성

Trouble Shooting

**금일 개발 과정 중, 문제를 해결한 경험이 있다면 기술해주세요.*

1. 문제 상황 파악
   - 이웃한 발판의 데이터 값에 따라 이동 불가 처리가 발생하지 않는 문제
2. 문제 발생 조건 파악
   - 초기 시점에 입력된 색/높이 데이터 상태가 계속 유지
   - 데이터 값 변동이 이웃 발판 데이터 감지 코드에 적용되지 않음
3. 결과 검증, 해결
   - 발판 색 변동에 이벤트를 추가
   - 색 변동 이벤트를 이웃 발판 데이터 감지 코드에 연결

그 외 과제 제출 (선택기입)

Github / Blog

**별도로 회고를 정리하는 깃헙, 블로그 게시글 링크가 있다면 첨부해주세요.*

- Github:
- Blog:
- Velog:

기술탐구

**프로젝트 작업을 위해 추가로 학습한 게시글이 있다면 링크를 첨부해주세요*

- (예시) 부트스트랩 설치방법: https://datamoney.tistory.com/149
- 링크 1:
- 링크 2: